

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Факультет информационных технологий
Кафедра параллельных вычислений**

ОТЧЕТ

О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ №2

«ВВЕДЕНИЕ В АРХИТЕКТУРУ x86/x86-64»

студента 2 курса, 24204 группы

Агапченко Владислава Вадимовича

Направление 09.03.01 – «Информатика и вычислительная техника»

Преподаватель:
к.т.н. А.Ю. Власенко

Новосибирск 2025

СОДЕРЖАНИЕ

ЦЕЛЬ.....	3
ЗАДАНИЕ.....	3
ОПИСАНИЕ РАБОТЫ.....	4
1 Изучить программную архитектуру x86/x86-64.....	4
2 Генерация ассемблерных листингов.....	4
3 Анализ ассемблерного кода.....	4
3.1 Сопоставление команд языка Си с машинными командами.....	4
3.2 Размещение переменных Си в программах на ассемблере.....	4
3.3 Оптимизационные преобразования.....	5
ЗАКЛЮЧЕНИЕ.....	8
Приложение 1. Код исходной программы <i>prog.c</i>	9
Приложение 2. Код программы <i>prog-O0.s</i>	10
Приложение 3. Код программы <i>prog-O3.s</i>	13

ЦЕЛЬ

Ознакомиться с программной архитектурой x86/86-64, синтаксисом ассемблера AT&T, проанализировать ассемблерный листинг программы соответствующей архитектуры.

ЗАДАНИЕ

- 1 Изучить программную архитектуру x86/86-64.
- 2 Для программы из практической работы 1 сгенерировать ассемблерные листинги (синтаксис AT&T, принятый в UNIX) для архитектуры x86/x86-64, используя уровни оптимизации O0 и O3.
- 3 Проанализировать полученные листинги и сделать:
 - 3.1 Сопоставить команды языка Си с машинными командами;
 - 3.2 Определить размещение переменных языка Си в программах на ассемблере;
 - 3.3 Выписать оптимизационные преобразования, выполненные компилятором;
 - 3.4 (опционально) сравнить различия в программах для архитектуры x86 и архитектуры x86-64.
- 4 Составить отчет по практической работе.

ОПИСАНИЕ РАБОТЫ

1 Изучить программную архитектуру x86/x86-64

Мной была изучена программная архитектура x86/x86-64, а именно:

- набор регистров,
- основные арифметико-логические команды,
- способы адресации памяти,
- способы передачи управления,
- работу со стеком,
- вызов подпрограмм,
- передачу параметров в подпрограммы и возврат результатов,
- работу с арифметическим сопроцессором,
- работу с векторными расширениями.

2 Генерация ассемблерных листингов

Код исходной программы представлен в приложении 1. Программы была скомпилирована на сайте godbolt.org, компилятором x86-64 gcc 15.2. Полный ассемблерный код для уровней оптимизаций O0 и O3 представлен в приложениях 2 и 3 соответственно.

3 Анализ ассемблерного кода

3.1 Сопоставление команд языка Си с машинными командами

Функции `double leibniz(long long)` соответствует метка `leibniz(long long)`, телу цикла — метка `.L3`, проверке условиям цикла и выходу из функции — метка `.L2`.

Основная функция `int main(int, char**)` соответствует метке `main`.

Вызовы функций соответствуют вызову операнда `call` с названием метки соответствующей функции, например `call leibniz`.

Параметры вызова функции сохраняются в регистры `rdi`. Возвращаемое значение сохраняется в регистр `rax`.

3.2 Размещение переменных Си в программах на ассемблере

Переменные сохраняются в область стека, соответствующую области стека функции. Обращение к данным происходит путем обращения к регистру `rbp` (начало области стека текущей функции) со смещением. В таблице 1 представлено соответствие переменных программы размещению данных в стеке на уровне оптимизации O0.

Си [type] [variable]	x86 ASM [shift](%rbp)
double leibniz(long long n)	
long long n	-40
double pi	-8
long long k	-16
long long sign	-24
int main(int argc, char **argv)	
int argc	-116
char **argv	-128
long clocks_per_sec	-8
long long n	-16
double pi	-24
long clocks	-32
double time	-40
struct tms start	-80
struct tms end	-112

Таблица 1 — Соответствие переменных размещению в стеке на уровне О0

3.3 Оптимизационные преобразования

На уровне оптимизации О3, компилятор применил следующие оптимизации:

1. Инлайнинг функции

В метке main нет вызова leibniz, вместо этого код встроен напрямую в метку main. Инициализация данных перед циклом производится перед меткой .L11, сама метка .L11 соответствует телу цикла и проверке условий цикла. Оптимизация представлена в листинге 1.

Листинг 1 — Инлайнинг функции leibniz

```

01 # Не оптимизировано
02 main:
03     ...
04 .L6:
05     ...
06     movq    -16(%rbp), %rax    # Запись входного параметра
07     movq    %rax, %rdi        # в регистр rdi
08
09     call    leibniz    # Вызов функции
10
11     movq    %xmm0, %rax        # Запись результата

```

```

12    movq    %rax, -24(%rbp)    # из регистра xmm0 в стек
13    ...
14
15 # Оптимизировано
16 main:
17    ...
18    leaq    1(%rbp,%rbp), %rcx # Код leibniz вставлен в main
19    movl    $1, %eax           # Инициализация переменных
20    pxor    %xmm0, %xmm0
21    movl    $1, %edx
22 .L11:
23    pxor %xmm1, %xmm1          # Тело цикла
24    pxor %xmm2, %xmm2
25    cvtsi2sdq %rdx, %xmm1
26    cvtsi2sdl %rax, %xmm2
27    divsd %xmm2, %xmm1
28    negq %rdx
29    addq $2, %rax
30    addsd %xmm1, %xmm0
31    cmpq %rax, %rcx
32    jne .L11
33 .L10:
34    ...

```

2. Отображение переменных на регистр процессора

На уровне оптимизации O0 переменные цикла π , n , k , sign хранились в стеке, однако на уровне оптимизации O3 хранятся в регистрах `xmm0`, `rsi`, `rax`, `edx` соответственно. Оптимизация представлена в листинге 2.

Листинг 2 — Отображение переменных на регистр процессора

```

01 # Не оптимизировано
02 leibniz:
03    ...
04    movq    %rdi, -40(%rbp)    # Инициализация n
05    pxor    %xmm0, %xmm0
06    movsd   %xmm0, -8(%rbp)    # Инициализация pi = 0
07    movq    $0, -16(%rbp)     # Инициализация k = 0
08    movq    $1, -24(%rbp)     # Инициализация sign = 1
09    ...
10 .L3:
11    # Далее работа со стеком
12    ...
13
14 # Оптимизировано
15 leibniz:
16    ...
17    leaq    1(%rbp, %rbp), %rcx # Инициализация n = 2k+1
18    movl    $1, %eax           # Инициализация k = 1
19    movl    $1, %edx           # Инициализация sign = 1
20    pxor    %xmm0, %xmm0      # Инициализация pi = 0

```

```
21      ...
22 .L3:
23      # Далее работа с регистрами, без обращения к стеку
```

За счет исключения обращений к стеку уменьшается время работы программы.

3. Переупорядочивание команд

В листинге 1 можно заметить изменения в инициализации переменных. Причиной тому является оптимизация вычислений цикла.

Итоговое количество итераций n вычисляется в регистр `rsx`. Счетчик итератора k инициализируется значением 1, изменяется с шагом 2. За счет таких изменений в теле цикла не приходится проводить вычисления для знаменателя $(2 * k + 1)$.

Аналог подобной оптимизации цикла на языке Си представлен в листинге 3.

Листинг 3 — Оптимизация цикла вычислений

```
01 // Не оптимизировано
02 for (long long k = 0, sign = 1; k < n; k++) {
03     pi += (double)sign / (2 * k + 1);
04     sign = -sign;
05 }
06
07 // Оптимизировано
08 for (long long k = 1, sign = 1; k <= 2*n + 1; k += 2) {
09     pi += (double)sign / k;
10     sign = -sign;
11 }
```

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы, мной была изучена программная архитектура x86-64, синтаксис ассемблера AT&T.

Проанализирована программа из практической работы 1 при компиляции в ассемблерный код на разных уровнях оптимизации, в том числе применяемые компилятором оптимизации на уровне ассемблерного кода.

Приложение 1. Код исходной программы *prog.c*

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <sys/times.h>
04 #include <unistd.h>
05
06 double leibniz(long long n) {
07     double pi = 0;
08
09     for (long long k = 0, sign = 1; k < n; k++) {
10         pi += (double)sign / (2 * k + 1);
11         sign = -sign;
12     }
13
14     return 4 * pi;
15 }
16
17 int main(int argc, char **argv) {
18     struct tms start, end;
19     long clocks_per_sec = sysconf(_SC_CLK_TCK);
20
21     if (argc < 2) {
22         printf("Missing argument N [long long]\n");
23         return EXIT_FAILURE;
24     }
25
26     long long n = atoll(argv[1]);
27
28     printf("Leibniz precision = %lld\n", n);
29
30     times(&start);
31     double pi = leibniz(n);
32     times(&end);
33
34     long clocks = end.tms_utime - start.tms_utime;
35     double time = (double)clocks / clocks_per_sec;
36
37     printf("Calculated PI = %.19g\n", pi);
38     printf("Calculation took %.5g sec.\n", time);
39
40     return EXIT_SUCCESS;
41 }
```

Приложение 2. Код программы prog-00.s

```
01 .section .rodata
02 .LC1:
03     .long    0
04     .long    1074790400
05 .LC2:
06     .string  "Missing argument N [long long]\n"
07 .LC3:
08     .string  "Leibniz precision = %lld\n"
09 .LC4:
10     .string  "Calculated PI = %.19g\n"
11 .LC5:
12     .string  "Calculation took %.5g sec.\n"
13
14 .section .text
15 leibniz:
16     pushq    %rbp
17     movq     %rsp, %rbp
18     movq     %rdi, -40(%rbp)
19     pxor     %xmm0, %xmm0
20     movsd    %xmm0, -8(%rbp)
21     movq     $0, -16(%rbp)
22     movq     $1, -24(%rbp)
23     jmp      .L2
24 .L3:
25     pxor     %xmm0, %xmm0
26     cvtsi2sdq    -24(%rbp), %xmm0
27     movq     -16(%rbp), %rax
28     addq     %rax, %rax
29     addq     $1, %rax
30     pxor     %xmm1, %xmm1
31     cvtsi2sdq    %rax, %xmm1
32     divsd    %xmm1, %xmm0
33     movsd    -8(%rbp), %xmm1
34     addsd    %xmm1, %xmm0
35     movsd    %xmm0, -8(%rbp)
36     negq     -24(%rbp)
37     addq     $1, -16(%rbp)
38 .L2:
39     movq     -16(%rbp), %rax
40     cmpq     -40(%rbp), %rax
41     jl       .L3
42     movsd    -8(%rbp), %xmm1
43     movsd    .LC1(%rip), %xmm0
44     mulsd    %xmm1, %xmm0
45     popq     %rbp
46     ret
47
48 .globl main
49 main:
```

```

50      pushq   %rbp
51      movq    %rsp, %rbp
52      addq    $-128, %rsp
53      movl    %edi, -116(%rbp)
54      movq    %rsi, -128(%rbp)
55      movl    $2, %edi
56      call    sysconf
57      movq    %rax, -8(%rbp)
58      cmpl    $1, -116(%rbp)
59      jg      .L6
60      movl    $.LC2, %edi
61      movl    $0, %eax
62      call    printf
63      movl    $1, %eax
64      jmp     .L8
65 .L6:
66      movq    -128(%rbp), %rax
67      addq    $8, %rax
68      movq    (%rax), %rax
69      movq    %rax, %rdi
70      call    atoll
71      movq    %rax, -16(%rbp)
72      movq    -16(%rbp), %rax
73      movq    %rax, %rsi
74      movl    $.LC3, %edi
75      movl    $0, %eax
76      call    printf
77      leaq    -80(%rbp), %rax
78      movq    %rax, %rdi
79      call    times
80      movq    -16(%rbp), %rax
81      movq    %rax, %rdi
82      call    leibniz
83      movq    %xmm0, %rax
84      movq    %rax, -24(%rbp)
85      leaq    -112(%rbp), %rax
86      movq    %rax, %rdi
87      call    times
88      movq    -112(%rbp), %rdx
89      movq    -80(%rbp), %rax
90      subq    %rax, %rdx
91      movq    %rdx, -32(%rbp)
92      pxor    %xmm0, %xmm0
93      cvtsi2sdq    -32(%rbp), %xmm0
94      pxor    %xmm1, %xmm1
95      cvtsi2sdq    -8(%rbp), %xmm1
96      divsd    %xmm1, %xmm0
97      movsd    %xmm0, -40(%rbp)
98      movq    -24(%rbp), %rax
99      movq    %rax, %xmm0

```

```
100      movl    $.LC4, %edi
101      movl    $1, %eax
102      call    printf
103      movq    -40(%rbp), %rax
104      movq    %rax, %xmm0
105      movl    $.LC5, %edi
106      movl    $1, %eax
107      call    printf
108      movl    $0, %eax
109 .L8:
110      leave
111      ret
```

Приложение 3. Код программы prog-O3.s

```
01 .section .rodata
02 .LC1:
03     .long    0
04     .long    1074790400
05 .LC2:
06     .string  "Missing argument N [long long]\n"
07 .LC3:
08     .string  "Leibniz precision = %lld\n"
09 .LC4:
10     .string  "Calculated PI = %.19g\n"
11 .LC5:
12     .string  "Calculation took %.5g sec.\n"
13
14 .section .text
15 leibniz:
16     testq    %rdi, %rdi
17     jle      .L4
18     leaq     1(%rdi,%rdi), %rcx
19     movl     $1, %eax
20     movl     $1, %edx
21     pxor     %xmm0, %xmm0
22 .L3:
23     pxor     %xmm1, %xmm1
24     pxor     %xmm2, %xmm2
25     cvtsi2sdq    %rdx, %xmm1
26     cvtsi2sdq    %rax, %xmm2
27     divsd     %xmm2, %xmm1
28     negq      %rdx
29     addq      $2, %rax
30     addsd     %xmm1, %xmm0
31     cmpq      %rax, %rcx
32     jne      .L3
33     mulsd     .LC1(%rip), %xmm0
34     ret
35 .L4:
36     pxor     %xmm0, %xmm0
37     ret
38
39 .globl main
40 main:
41     pushq     %r12
42     movq      %rsi, %r12
43     pushq     %rbp
44     movl      %edi, %ebp
45     movl      $2, %edi
46     pushq     %rbx
47     subq      $80, %rsp
48     call      sysconf
49     cmpl      $1, %ebp
```

```

50     jle     .L15
51     movq    8(%r12), %rdi
52     xorl    %esi, %esi
53     movl    $10, %edx
54     movq    %rax, %rbx
55     call    strtoll
56     movl    $.LC3, %edi
57     movq    %rax, %rbp
58     movq    %rax, %rsi
59     xorl    %eax, %eax
60     call    printf
61     leaq    16(%rsp), %rdi
62     call    times
63     testq   %rbp, %rbp
64     jle     .L12
65     leaq    1(%rbp,%rbp), %rcx
66     movl    $1, %eax
67     pxor    %xmm0, %xmm0
68     movl    $1, %edx
69 .L11:
70     pxor    %xmm1, %xmm1
71     pxor    %xmm2, %xmm2
72     cvtsi2sdq    %rdx, %xmm1
73     cvtsi2sdq    %rax, %xmm2
74     divsd    %xmm2, %xmm1
75     negq     %rdx
76     addq     $2, %rax
77     addsd    %xmm1, %xmm0
78     cmpq     %rax, %rcx
79     jne     .L11
80 .L10:
81     mulsd    .LC1(%rip), %xmm0
82     leaq     48(%rsp), %rdi
83     movsd    %xmm0, 8(%rsp)
84     call    times
85     movq     48(%rsp), %rax
86     pxor     %xmm1, %xmm1
87     subq     16(%rsp), %rax
88     pxor     %xmm2, %xmm2
89     cvtsi2sdq    %rax, %xmm1
90     movsd    8(%rsp), %xmm0
91     movl     $.LC4, %edi
92     cvtsi2sdq    %rbx, %xmm2
93     divsd    %xmm2, %xmm1
94     movl     $1, %eax
95     movsd    %xmm1, (%rsp)
96     call    printf
97     movsd    (%rsp), %xmm0
98     movl     $.LC5, %edi
99     movl     $1, %eax

```

100	call	printf
101	xorl	%eax, %eax
102	.L7:	
103	addq	\$80, %rsp
104	popq	%rbx
105	popq	%rbp
106	popq	%r12
107	ret	
108	.L15:	
109	movl	\$.LC2, %edi
110	xorl	%eax, %eax
111	call	printf
112	movl	\$1, %eax
113	jmp	.L7
114	.L12:	
115	pxor	%xmm0, %xmm0
116	jmp	.L10